LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 INHERITANCE



NAMA : Jeremy Marcello Waani

NIM : 2311110003

KELAS : PS1DS-05

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. **Dasar Teori**

Inheritance atau pewarisan adalah mekanisme pembentukan class turunan dari class induk menggunakan kata kunci extends. Class turunan dapat memakai anggota induk yang dapat diakses, menambah atribut, dan menyesuaikan perilaku. Java mendukung satu class induk langsung, tetapi pewarisan dapat berlangsung bertingkat, misalnya Model, ModelKlasifikasi, dan ModelKlasifikasiDeepLearning.

Enkapsulasi tetap berlaku dalam pewarisan. Atribut private tidak dapat diakses langsung dari class turunan, sehingga pembacaan data dilakukan melalui getter. Constructor tidak diwariskan. Pemanggilan super(...) menjalankan constructor induk untuk mengisi data dasar, sedangkan this digunakan untuk merujuk atribut objek saat namanya sama dengan parameter.

Overriding dilakukan ketika class turunan mendefinisikan kembali method instance dengan nama dan daftar parameter yang sama serta tipe hasil yang sesuai. Anotasi @Override membantu compiler memeriksa kesesuaiannya. Pemanggilan super.namaMethod() menjalankan implementasi induk, sehingga class turunan dapat menambahkan proses tanpa mengulang seluruh kode.

Overloading menyediakan beberapa method bernama sama dengan daftar parameter berbeda. Perbedaannya dapat berupa jumlah atau tipe parameter, bukan hanya tipe nilai kembali. Pada praktikum ini, hitungBiaya(), hitungBiaya(int epoch), dan hitungBiaya(int epoch, int ukuranBatch) dipilih berdasarkan argumen yang diberikan. Method overload yang diwariskan tetap dapat digunakan.

Polymorphism memungkinkan referensi bertipe induk menyimpan objek turunan. Ketika method yang di-override dipanggil melalui referensi tersebut, implementasi yang dijalankan mengikuti jenis objek sebenarnya. Array Model[] karena itu dapat menampung model regresi, klasifikasi, dan deep learning, lalu memproses semuanya dengan satu perulangan.

Konsep	**Kata kunci atau bentuk**	**Penerapan**
Pewarisan	extends dan super(...)	Membentuk class turunan dan mengisi data induk
Overriding	@Override	Menyesuaikan biaya sesuai jenis model
Overloading	Nama sama, parameter berbeda	Menghitung biaya biasa, per epoch, dan per batch
Polymorphism	Array Model[]	Memproses tiga objek turunan melalui tipe induk

B. Guided

1. Pewarisan pada class A dan B

guided/gd1/A.java

```
package guided.gd1;

public class A {
    int x;
    int y;

    void tampilXY() {
        System.out.println("Nilai x = " + x + ", y = " + y);
    }
}
```

guided/gd1/B.java

```
package guided.gd1;

public class B extends A {
    int z;
    int hasil;

    void jumlahXYZ() {
        hasil = (x + y + z);

        System.out.println("Jumlah (x+y+z) = " + hasil);
    }
}
```

guided/gd1/Demo.java

```
package guided.gd1;

public class Demo {
    public static void main(String[] args) {
        A superclass = new A();
        B subclass = new B();

        superclass.x = 3;
        superclass.y = 4;

        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;

        System.out.println("Subclass: ");
        subclass.jumlahXYZ();
        subclass.tampilXY();
    }
}
```

Output

```
java -cp target/classes guided.gd1.Demo
```

```
Superclass:
Nilai x = 3, y = 4
Subclass:
Jumlah (x+y+z) = 8
Nilai x = 1, y = 2
```

Gambar 1 Output guided.gd1.Demo

2. **Penggunaan super pada Employee dan Manager**

guided/gd2/Employee.java

```
package guided.gd2;

public class Employee {
    private String name;
    String departemen;

    public Employee(String name) {
        this.name = name;
    }

    public void tampilNama() {
        System.out.println("nama: " + name);
    }
}
```

guided/gd2/Manager.java

```
package guided.gd2;

public class Manager extends Employee{
    private String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("alamat: " + alamat);
        System.out.println("departemen: " + super.departemen);
    }
}
```

guided/gd2/Demo.java

```
package guided.gd2;

public class Demo {
    public static void main(String[] args){
        Manager adi = new Manager("Adi", "Sukabirus");
        adi.departemen = "SDM";
```

guided/gd2/Demo.java

```
        adi.tampilInformasi();
    }
}
```

Output

```
java -cp target/classes guided.gd2.Demo
```

```
nama: Adi
alamat: Sukabirus
departemen: SDM
```

Gambar 2 Output guided.gd2.Demo

3. ### Overloading pada class Lagu

guided/gd3/Lagu.java

```
package guided.gd3;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    void tampilkan() {
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}
```

guided/gd3/Main.java

```
package guided.gd3;

public class Main {
    public static void main(String[] args) {
        Lagu d, e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("Lagu 2", "aku");

        d.tampilkan();
        e.tampilkan();
    }
}
```

Output

```
java -cp target/classes guided.gd3.Main
```

```
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : Lagu 2, pencipta : aku
```

Gambar 3 Output guided.gd3.Main

4. **Overriding pada class A dan B**

guided/gd4/A.java

```
package guided.gd4;

public class A {
    public void tampilkan() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

guided/gd4/B.java

```
package guided.gd4;

class B extends A {
    public void tampilkan() {
        super.tampilkan(); // Method milik superclas dipanggil
        System.out.println("Method milik class B dipanggil...");
    }
}
```

guided/gd4/Main.java

```
package guided.gd4;

class Main {
    public static void main(String[] args) {
        B subOb = new B();
        subOb.tampilkan();
    }
}
```

Output

```
java -cp target/classes guided.gd4.Main
```

```
Method milik class A dipanggil...
Method milik class B dipanggil...
```

Gambar 4 Output guided.gd4.Main

C. **Unguided**

1. **Estimasi Biaya Komputasi Model Machine Learning**

Soal

Sebuah tim machine learning membutuhkan program Java untuk membandingkan estimasi biaya komputasi model regresi, klasifikasi, dan klasifikasi deep learning. Semua atribut harus private. Setiap class ditulis pada file .java terpisah dan ditempatkan dalam package unguided.

Class induk menyimpan nama model dan jumlah data latih, menyediakan constructor, getter jumlah data, serta method informasi. Method penghitung biaya dibuat dalam dua bentuk: tanpa parameter menghasilkan jumlah data latih x 2, sedangkan satu parameter epoch menghasilkan jumlah data latih x epoch.

Model regresi menambah jumlah fitur dan menghitung biaya dasar ditambah jumlah fitur x 10. Model ini juga menyediakan overload dua parameter dengan rumus jumlah data latih x epoch / ukuran batch. Model klasifikasi menambah jumlah kelas dan menghitung biaya induk dikalikan jumlah kelas tanpa meng-override versi satu parameter.

Model klasifikasi deep learning harus mewarisi model klasifikasi dan menambah jumlah layer. Biayanya adalah biaya klasifikasi ditambah jumlah layer x 200. Semua turunan memakai constructor induk dan meng-override method informasi dengan terlebih dahulu memanggil versi induknya.

Main membuat tiga objek berikut, menyimpannya dalam satu array bertipe induk, menampilkan informasi dan biaya dengan perulangan, lalu menjumlahkan biaya. Terakhir, program menampilkan biaya 10 epoch Neural Network yang diambil dari array, biaya 10 epoch Regresi Linier, serta biaya regresi 10 epoch dengan batch 50.

Nama model	Data latih	Atribut tambahan
Regresi Linier	500	4 fitur
Decision Tree	800	3 kelas
Neural Network	1000	5 kelas dan 3 layer

Jawaban

unguided/Model.java

```
package unguided;

public class Model {
    private String namaModel;
    private int jumlahDataLatih;

    public Model(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public void tampilInformasi() {
        System.out.println("Model: " + namaModel + " | Data latih: " +
jumlahDataLatih);
    }

    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

unguided/ModelRegresi.java

```
package unguided;

public class ModelRegresi extends Model {
    private int jumlahFitur;

    public ModelRegresi(String namaModel, int jumlahDataLatih, int jumlahFitur) {
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    @Override
    public void tampilInformasi() {
        super.tampilInformasi();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    public int hitungBiaya(int epoch, int ukuranBatch) {
        return getJumlahDataLatih() * epoch / ukuranBatch;
    }
}
```

unguided/ModelKlasifikasi.java

```
package unguided;

public class ModelKlasifikasi extends Model {
    private int jumlahKelas;

    public ModelKlasifikasi(String namaModel, int jumlahDataLatih, int
jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    @Override
    public void tampilInformasi() {
        super.tampilInformasi();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

unguided/ModelKlasifikasiDeepLearning.java

```
package unguided;

public class ModelKlasifikasiDeepLearning extends ModelKlasifikasi {
    private int jumlahLayer;

    public ModelKlasifikasiDeepLearning(String namaModel, int jumlahDataLatih,
            int jumlahKelas, int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilInformasi() {
        super.tampilInformasi();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

unguided/Main.java

```
package unguided;

public class Main {
    public static void main(String[] args) {
        ModelRegresi regresi = new ModelRegresi("Regresi Linier", 500, 4);
        ModelKlasifikasi klasifikasi = new ModelKlasifikasi("Decision Tree", 800,
3);
```

unguided/Main.java

```
        ModelKlasifikasiDeepLearning deepLearning = new
ModelKlasifikasiDeepLearning(
                "Neural Network", 1000, 5, 3);

        Model[] daftarModel = {regresi, klasifikasi, deepLearning};
        int totalBiaya = 0;

        for (Model model : daftarModel) {
            model.tampilInformasi();
            int biaya = model.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            totalBiaya += biaya;
        }

        System.out.println("Total biaya: " + totalBiaya);
        System.out.println("Biaya 10 epoch (Neural Network): "
                + daftarModel[2].hitungBiaya(10));
        System.out.println("Biaya 10 epoch (Regresi Linier): " +
regresi.hitungBiaya(10));
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): "
                + regresi.hitungBiaya(10, 50));
    }
}
```

Output

```
java -cp target/classes unguided.Main
```

```
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

Gambar 5 Output unguided.Main

Penjelasan program

Model menjadi class induk yang menyimpan namaModel dan jumlahDataLatih sebagai atribut private. Constructor mengisi keduanya, sedangkan getJumlahDataLatih() menyediakan akses baca untuk class turunan. Method tampilInformasi() mencetak identitas model. Dua method hitungBiaya() dibedakan oleh ada atau tidaknya parameter epoch.

ModelRegresi mewarisi Model dan menambah jumlahFitur. Constructor memanggil super(namaModel, jumlahDataLatih). Method tampilInformasi() memanggil versi induk, kemudian mencetak jumlah fitur. Override hitungBiaya() menambah biaya fitur, sedangkan hitungBiaya(int epoch, int ukuranBatch) memakai getter untuk menghitung biaya per batch. Objek regresi memiliki tiga versi penghitung biaya karena versi satu parameter tetap diwarisi.

ModelKlasifikasi juga mewarisi Model, tetapi menambah jumlahKelas. Override hitungBiaya() mengalikan super.hitungBiaya() dengan jumlah kelas. Method hitungBiaya(int epoch) tidak ditulis ulang, sehingga perhitungannya tetap mengikuti Model dan tidak dikalikan jumlah kelas.

ModelKlasifikasiDeepLearning mewarisi ModelKlasifikasi. Pemanggilan super.hitungBiaya() pada class ini sudah menghasilkan biaya klasifikasi, lalu ditambah jumlahLayer x 200. Pemanggilan super.tampilInformasi() juga mencetak informasi umum dan jumlah kelas sebelum jumlah layer ditampilkan. Susunan ini menunjukkan pewarisan bertingkat.

Main membuat ketiga objek dan menyimpannya dalam daftarModel bertipe Model[]. Pada perulangan for-each, pemanggilan tampilInformasi() dan hitungBiaya() mengikuti jenis objek yang tersimpan. Setiap biaya dijumlahkan ke totalBiaya. Pemanggilan daftarModel[2].hitungBiaya(10) menunjukkan bahwa overload milik Model tetap tersedia pada objek deep learning melalui referensi induk.

Analisis hasil perhitungan

Biaya tanpa parameter dihitung melalui rangkaian pemanggilan super sesuai jenis model. Hasil untuk ketiga objek adalah sebagai berikut.

Model	Perhitungan	Biaya
Regresi Linier	500 x 2 + 4 x 10	1040
Decision Tree	800 x 2 x 3	4800
Neural Network	1000 x 2 x 5 + 3 x 200	10600

Total biaya adalah 1040 + 4800 + 10600 = 16440. Biaya 10 epoch Neural Network bernilai 1000 x 10 = 10000 karena menggunakan overload satu parameter yang diwarisi dari Model. Biaya 10 epoch Regresi Linier adalah 500 x 10 = 5000, sedangkan versi batch menghasilkan 500 x 10 / 50 = 100. Perhitungan memakai int, sehingga pembagian pada versi batch merupakan pembagian bilangan bulat.

Pengujian program

Seluruh 17 file Java pada proyek berhasil dikompilasi menggunakan JDK 27 dengan target Java 26. Empat program guided dan unguided.Main berhasil dijalankan. Output unguided dibandingkan baris demi baris dengan contoh pada soal dan menghasilkan nilai yang sama, termasuk total biaya serta ketiga pemanggilan overload tambahan.

D. **Kesimpulan**

Praktikum Modul 6 menunjukkan bahwa inheritance membantu menggunakan kembali atribut dan perilaku umum melalui class induk. Constructor dengan super mengisi data induk, sedangkan atribut private tetap dibaca melalui method yang disediakan. Overriding menyesuaikan perilaku pada class turunan, sementara overloading menyediakan beberapa cara pemanggilan method bernama sama.

Pada unguided, pewarisan bertingkat menghubungkan Model, ModelKlasifikasi, dan ModelKlasifikasiDeepLearning. Array Model[] memungkinkan tiga jenis model diproses dalam satu perulangan melalui polymorphism. Hasil program sesuai dengan ketentuan soal: biaya regresi 1040, klasifikasi 4800, deep learning 10600, dan total 16440. Pemanggilan biaya per epoch dan per batch juga menghasilkan 10000, 5000, dan 100.